ДИСЦИПЛИНА **Конфигурационное управление**

(полное наименование дисциплины без сокращений)

ИНСТИТУТ **Информационных технологий**

КАФЕДРА **Корпоративных информационных систем**

(полное наименование кафедры)

ВИД УЧЕБНОГО МАТЕРИАЛА **Задание для текущего контроля**

(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **П.Н. Советов**

(фамилия, имя, отчество)

СЕМЕСТР **3 семестр (осенний) 2026/2027 учебного года**

(указать семестр обучения, учебный год)

Конфигурационное управление

Сборник практических работ № 2

ИКБО-21-25

РТУ МИРЭА – 2026

Оглавление

О практических работах 4
Перечень нарушений оформления git-репозитория 5
Вариант №1 8
Вариант №2 11
Вариант №3 14
Вариант №4 17
Вариант №5 20
Вариант №6 23
Вариант №7 26
Вариант №8 29
Вариант №9 32
Вариант №10 35
Вариант №11 38
Вариант №12 41
Вариант №13 44
Вариант №14 47
Вариант №15 50
Вариант №16 53
Вариант №17 56
Вариант №18 59
Вариант №19 62
Вариант №20 65
Вариант №21 68
Вариант №22 71
Вариант №23 74
Вариант №24 77
Вариант №25 80

Вариант №26 83
Вариант №27 86
Вариант №28 89
Вариант №29 92
Вариант №30 95
Вариант №31 98
Вариант №32 101
Вариант №33 104
Вариант №34 107
Вариант №35 110
Вариант №36 113
Вариант №37 116
Вариант №38 119
Вариант №39 122
Вариант №40 125

О практических работах

Практические работы (ПР) состоят из нескольких этапов. ПР выполняются очно и защита каждого этапа происходит на семинарских занятиях. Этапы работы над ПР сохраняются в публично доступном git-репозитории. Каждый этап разработки ПР должен быть отражен в истории коммитов с детальными сообщениями. Студент самостоятельно выбирает язык реализации.

Документация оформляется в виде readme.md, который содержит:

1. Общее описание.
2. Описание всех функций и настроек.
3. Описание команд для сборки проекта и запуска тестов.
4. Примеры использования.

Список публичных git-сервисов для репозиториев:

1. github.com
2. gitea.com
3. gitlab.com
4. gitflic.ru
5. hub.mos.ru
6. gitverse.ru
7. gitee.com

Репозиторий должен быть оформлен стандартно и не должен содержать типичных нарушений, перечисленных в разделе «Перечень нарушений оформления git-репозитория».

Перечень нарушений оформления git-репозитория

№	Описание правила
Структура репозитория	
С1.	В репозитории отсутствует файл README.md
С2.	В репозитории отсутствует файл .gitignore
С3.	В репозитории отсутствует папка с исходным кодом src
С4.	В репозитории отсутствует папка с тестами tests
С5.	В репозитории отсутствует скрипт запуска run.sh, run.bat или Makefile
С6.	Коммиты не соответствуют спецификации Conventional/Scoped Commits
Сгенерированные файлы	
Г1.	Репозиторий содержит архивы
Г2.	Репозиторий содержит бинарные файлы
Г3.	Репозиторий содержит служебные файлы языка Python
Г4.	Репозиторий содержит артефакты сборки
Г5.	Репозиторий содержит файлы с настройками редактора кода
Рецензирование	
Р1.	Репозиторий содержит слишком большие файлы (>1000 строк)
Р2.	Код на языке Python содержит слишком длинные строки (>80 символов)
Р3.	Код на языке JavaScript содержит слишком длинные строки (>80 символов)
Р4.	Код на языке Kotlin содержит слишком длинные строки (>100 символов)
Р5.	Код на языке Rust содержит слишком длинные строки (>100 символов)
Р6.	Код в репозитории содержит слишком длинные строки (>120 символов)
Функции	
Ф1.	Код на языке Python содержит слишком длинные функции (>40 строк)
Ф2.	Код на языке JavaScript содержит слишком длинные функции (>40 строк)
Ф3.	Код на языке C# содержит слишком длинные функции (>40 строк)
Ф4.	Код на языке Java содержит слишком длинные функции (>40 строк)
Ф5.	Код на языке Go содержит слишком длинные функции (>40 строк)
Ф6.	Код на языке Kotlin содержит слишком длинные функции (>40 строк)
Ф7.	Код на языке Rust содержит слишком длинные функции (>40 строк)
Аргументы функций	
А1.	Код на Python содержит функции с большим числом аргументов (>7)
А2.	Код на JavaScript содержит функции с большим числом аргументов (>7)

№	Описание правила
А3.	Код на C# содержит функции с большим числом аргументов (>7)
А4.	Код на Java содержит функции с большим числом аргументов (>7)
А5.	Код на Go содержит функции с большим числом аргументов (>7)
А6.	Код на Kotlin содержит функции с большим числом аргументов (>7)
А7.	Код на Rust содержит функции с большим числом аргументов (>7)
Комментарии вместо документации	
К1.	Код на Python содержит комментарии вместо документации docstring
К2.	Код на JavaScript содержит комментарии вместо документации JSDoc
К3.	Код на C# содержит комментарии вместо документации XMLDoc
К4.	Код на Java содержит комментарии вместо документации Javadoc
К5.	Код на Go содержит комментарии вместо документации Godoc
К6.	Код на Kotlin содержит комментарии вместо документации KDoc
К7.	Код на Rust содержит комментарии вместо документации rustdoc
Цикломатическая сложность кода	
Ц1.	Цикломатическая сложность кода на языке Python превышена (>10)
Ц2.	Цикломатическая сложность кода на языке JavaScript превышена (>10)
Ц3.	Цикломатическая сложность кода на языке C# превышена (>10)
Ц4.	Цикломатическая сложность кода на языке Java превышена (>10)
Ц5.	Цикломатическая сложность кода на языке Go превышена (>10)
Ц6.	Цикломатическая сложность кода на языке Kotlin превышена (>10)
Ц7.	Цикломатическая сложность кода на языке Rust превышена (>10)
Магические числа	
М1.	Код на Python содержит магические числовые литералы в сравнениях
М2.	Код на JavaScript содержит магические числовые литералы в сравнениях
М3.	Код на C# содержит магические числовые литералы в сравнениях
М4.	Код на Java содержит магические числовые литералы в сравнениях
М5.	Код на Go содержит магические числовые литералы в сравнениях
М6.	Код на Kotlin содержит магические числовые литералы в сравнениях
М7.	Код на Rust содержит магические числовые литералы в сравнениях
Имена идентификаторов	
И1.	Код на Python содержит имена, не соответствующие стандарту PEP8
И2.	Код на JavaScript содержит имена, нарушающие соглашения ESLint
И3.	Код на C# содержит имена, нарушающие соглашения .NET Runtime

№	Описание правила
И4.	Код содержит имена, нарушающие стандартные соглашения Java
И5.	Код содержит имена, нарушающие стандартные соглашения Go
И6.	Код содержит имена, нарушающие стандартные соглашения Kotlin
И7.	Код содержит имена, нарушающие стандартные соглашения Rust

Вариант №1

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов .NET (NuGet).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS с рекурсией.
2. Корректно обработать случаи наличия циклических зависимостей.
3. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Mermaid.
2. Сохранить изображение графа в файле формата PNG.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.

5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №2

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Имя сгенерированного файла с изображением графа.
   - Режим вывода зависимостей в формате ASCII-дерева.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов JavaScript (npm).
2. Информацию необходимо получить для заданной пользователем версии пакета.
3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.

4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS без рекурсии.
2. Корректно обработать случаи наличия циклических зависимостей.
3. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм D2.
2. Сохранить изображение графа в файле формата PNG.

3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №3

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата INI.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Имя сгенерированного файла с изображением графа.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Ubuntu (apt).
2. Информацию необходимо получить для заданной пользователем версии пакета.
3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS с рекурсией.
2. Корректно обработать случаи наличия циклических зависимостей.
3. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Graphviz.
2. Сохранить изображение графа в файле формата PNG.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.

4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №4

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата CSV.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Rust (Cargo).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм D2.
2. Вывести на экран изображение графа.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №5

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
   - Режим вывода зависимостей в формате ASCII-дерева.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Alpine Linux (apk).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS с рекурсией.
2. Корректно обработать случаи наличия циклических зависимостей.
3. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Mermaid.
2. Сохранить изображение графа в файле формата SVG.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.

5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №6

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата XML.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Alpine Linux (apk).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS без рекурсии.
2. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Graphviz.
2. Вывести на экран изображение графа.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.

4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №7

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата YAML.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Имя сгенерированного файла с изображением графа.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов JavaScript (npm).
2. Информацию необходимо получить для заданной пользователем версии пакета.

3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS с рекурсией.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм D2.
2. Сохранить изображение графа в файле формата PNG.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №8

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата TOML.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Java (Maven).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм PlantUML.
2. Вывести на экран изображение графа.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.

4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №9

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата TOML.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Java (Maven).
2. Информацию необходимо получить для заданной пользователем версии пакета.
3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS с рекурсией.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Mermaid.
2. Вывести на экран описание графа на языке диаграмм.

3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №10

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата INI.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов JavaScript (npm).
2. Информацию необходимо получить для заданной пользователем версии пакета.

3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Mermaid.
2. Вывести на экран описание графа на языке диаграмм.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №11

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата XML.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Имя сгенерированного файла с изображением графа.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Alpine Linux (apk).
2. Информацию необходимо получить для заданной пользователем версии пакета.
3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.

4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS с рекурсией.
2. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Mermaid.

2. Сохранить изображение графа в файле формата PNG.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №12

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов .NET (NuGet).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS с рекурсией.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм PlantUML.
2. Сохранить изображение графа в файле формата SVG.

3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №13

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата YAML.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Python (pip).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм PlantUML.
2. Вывести на экран описание графа на языке диаграмм.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №14

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата YAML.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Имя сгенерированного файла с изображением графа.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Rust (Cargo).
2. Информацию необходимо получить для заданной пользователем версии пакета.

3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS с рекурсией.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Graphviz.
2. Сохранить изображение графа в файле формата SVG.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №15

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Python (pip).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм D2.

2. Сохранить изображение графа в файле формата SVG.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №16

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата XML.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Java (Maven).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS без рекурсии.
2. Корректно обработать случаи наличия циклических зависимостей.
3. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм D2.
2. Сохранить изображение графа в файле формата PNG.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.

5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №17

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Java (Maven).
2. Информацию необходимо получить для заданной пользователем версии пакета.
3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.

4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS с рекурсией.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм PlantUML.
2. Вывести на экран описание графа на языке диаграмм.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №18

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата INI.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Ubuntu (apt).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS с рекурсией.
2. Корректно обработать случаи наличия циклических зависимостей.
3. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм D2.
2. Вывести на экран описание графа на языке диаграмм.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.

5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №19

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Ubuntu (apt).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS с рекурсией.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Mermaid.
2. Сохранить изображение графа в файле формата SVG.

3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №20

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Имя сгенерированного файла с изображением графа.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов .NET (NuGet).
2. Информацию необходимо получить для заданной пользователем версии пакета.

3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS с рекурсией.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Graphviz.
2. Сохранить изображение графа в файле формата SVG.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №21

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Rust (Cargo).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS без рекурсии.
2. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Graphviz.
2. Сохранить изображение графа в файле формата PNG.

3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №22

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата JSON.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Alpine Linux (apk).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS с рекурсией.
2. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм D2.
2. Сохранить изображение графа в файле формата PNG.

3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №23

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Python (pip).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.

3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм D2.
2. Сохранить изображение графа в файле формата SVG.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №24

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов .NET (NuGet).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS без рекурсии.
2. Корректно обработать случаи наличия циклических зависимостей.
3. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм D2.
2. Сохранить изображение графа в файле формата SVG.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.

5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №25

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата JSON.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Режим вывода зависимостей в формате ASCII-дерева.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Alpine Linux (apk).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS с рекурсией.
2. Корректно обработать случаи наличия циклических зависимостей.
3. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм PlantUML.
2. Вывести на экран описание графа на языке диаграмм.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.

5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №26

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата XML.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Java (Maven).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм PlantUML.
2. Вывести на экран описание графа на языке диаграмм.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.

4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №27

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов .NET (NuGet).
2. Информацию необходимо получить для заданной пользователем версии пакета.
3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм D2.
2. Вывести на экран изображение графа.

3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №28

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Ubuntu (apt).
2. Информацию необходимо получить для заданной пользователем версии пакета.
3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.

4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS с рекурсией.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Mermaid.

2. Вывести на экран описание графа на языке диаграмм.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №29

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата CSV.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов JavaScript (npm).
2. Информацию необходимо получить для заданной пользователем версии пакета.
3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.

4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS с рекурсией.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм PlantUML.
2. Вывести на экран изображение графа.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №30

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Java (Maven).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS с рекурсией.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Mermaid.
2. Вывести на экран описание графа на языке диаграмм.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.

4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №31

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Python (pip).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS с рекурсией.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Graphviz.
2. Вывести на экран описание графа на языке диаграмм.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.

4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №32

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Java (Maven).
2. Информацию необходимо получить для заданной пользователем версии пакета.
3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Graphviz.
2. Вывести на экран описание графа на языке диаграмм.

3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №33

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Alpine Linux (apk).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Mermaid.
2. Сохранить изображение графа в файле формата SVG.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.

4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №34

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата YAML.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Rust (Cargo).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS с рекурсией.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Корректно обработать случаи наличия циклических зависимостей.
4. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Graphviz.
2. Сохранить изображение графа в файле формата PNG.

3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №35

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата JSON.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Java (Maven).
2. Информацию необходимо получить для заданной пользователем версии пакета.

3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Mermaid.
2. Вывести на экран изображение графа.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №36

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата INI.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Режим вывода зависимостей в формате ASCII-дерева.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов JavaScript (npm).
2. Информацию необходимо получить для заданной пользователем версии пакета.
3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.

5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS без рекурсии.
2. Корректно обработать случаи наличия циклических зависимостей.
3. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Graphviz.
2. Вывести на экран изображение графа.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.

4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №37

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Имя сгенерированного файла с изображением графа.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Rust (Cargo).
2. Информацию необходимо получить для заданной пользователем версии пакета.

3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм D2.
2. Сохранить изображение графа в файле формата PNG.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №38

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата JSON.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов JavaScript (npm).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS без рекурсии.
2. Корректно обработать случаи наличия циклических зависимостей.
3. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран обратных зависимостей для заданного пакета. Это те пакеты, которые зависят от данного пакета. Использовать алгоритм обхода из предыдущего этапа.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм Graphviz.
2. Вывести на экран описание графа на языке диаграмм.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.

5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №39

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров являются опции командной строки.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Версия пакета.
   - Имя сгенерированного файла с изображением графа.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Python (pip).
2. Информацию необходимо получить для заданной пользователем версии пакета.

3. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.
4. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом DFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм PlantUML.
2. Сохранить изображение графа в файле формата SVG.
3. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
4. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Вариант №40

Разработать инструмент визуализации графа зависимостей для менеджера пакетов. Готовые средства (менеджеры пакетов, библиотеки) для получения зависимостей использовать нельзя.

Этап 1. Минимальный прототип с конфигурацией

Цель: создать минимальное CLI-приложение и сделать его настраиваемым.

Требования:

1. Источником настраиваемых пользователем параметров является конфигурационный файл формата YAML.
2. К настраиваемым параметрам относятся:
   - Имя анализируемого пакета.
   - URL-адрес репозитория или путь к файлу тестового репозитория.
   - Режим работы с тестовым репозиторием.
   - Имя сгенерированного файла с изображением графа.
   - Режим вывода зависимостей в формате ASCII-дерева.
   - Максимальная глубина анализа зависимостей.
   - Подстрока для фильтрации пакетов.
3. (только для этого этапа) При запуске приложения вывести все параметры, настраиваемые пользователем, в формате ключ-значение.
4. Реализовать и продемонстрировать обработку ошибок для всех параметров.
5. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 2. Сбор данных

Цель: реализовать основную логику получения данных о зависимостях для их дальнейшего анализа и визуализации. Запрещено пользоваться менеджерами пакетов и сторонними библиотеками для получения информации о зависимостях пакетов.

Требования:

1. Использовать формат пакетов Java (Maven).
2. Извлечь информацию о прямых зависимостях заданного пользователем пакета, используя URL-адрес репозитория.

3. (только для этого этапа) Вывести на экран все прямые зависимости заданного пользователем пакета.
4. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 3. Основные операции

Цель: построить граф зависимостей (с учетом транзитивности) и выполнить основные операции над ним.

Требования:

1. Получение графа зависимостей реализовать алгоритмом BFS без рекурсии.
2. Проводить анализ с учетом максимальной глубины, заданной пользователем.
3. Не учитывать при анализе пакеты, имя которых содержит заданную пользователем подстроку.
4. Корректно обработать случаи наличия циклических зависимостей.
5. Поддержать режим тестирования. Вместо URL реального репозитория, дать возможность пользователю указать путь к файлу описания графа репозитория, где пакеты называются большими латинскими буквами. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 4. Дополнительные операции

Цель: выполнить дополнительные операции над графом зависимостей.

Требования:

1. (только для этого этапа) Поддержать режим вывода на экран порядка загрузки зависимостей для заданного пакета. Сравнить результаты с реальным менеджером пакетов. Если есть расхождения в результатах, объяснить их наличие.
2. Продемонстрировать функциональность этого этапа на различных случаях работы с тестовым репозиторием.
3. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.

Этап 5. Визуализация

Цель: получить графическое представление графа зависимостей.

Требования:

1. Сформировать текстовое представление графа зависимостей на языке диаграмм PlantUML.
2. Сохранить изображение графа в файле формата SVG.
3. Если задан соответствующий параметр, вывести на экран зависимости в виде ASCII-дерева.
4. Продемонстрировать примеры визуализации зависимостей для трех различных пакетов.
5. Сравнить результаты с выводом штатных инструментов визуализации для выбранного менеджера пакетов. Если есть расхождения в результатах, объяснить их наличие.
6. Результат выполнения этапа сохранить в репозиторий стандартно оформленным коммитом.